| | |
|---|---|
| **Name:** Ruzyl Bryan V. Amora | **Date:** March 5 2026 |
| **Section:** CPE31S1 | **Instructor:** Engr Neal Matira |

| Phase | Activity | Code and Output |
|---|---|---|
| **Extract** | 1. Import necessary libraries like Pandas, KaggleHub. | **Code:** import pandas as pd, import kagglehub |
| | 2. Download the RT-IoT2022 dataset using Kaggle API. | **Code:** path = kagglehub.dataset_download("supplejade/rt-iot2022real-time-internet-of-things") |
| | 3. Load the flat CSV file into a Pandas DataFrame. | **Code:** df = pd.read_csv(f"{path}/RT_IOT2022.csv") |
| | 4. Verify data retrieval by checking shape and the first few rows. | **Output:** Extraction Successful. Dataset Shape: (123117, 85) followed by the df.head() table. |

| **Phase 1 Output** |
|---|

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readth
  from .autonotebook import tqdm as notebook_tqdm
Path to dataset files: /Users/macbookpro/.cache/kagglehub/datasets/supplejade/rt-iot2022real-time-internet-of-things/versions/3
Extraction Successful. Dataset Shape: (123117, 85)
```

| | no | id.orig_p | id.resp_p | proto | service | flow_duration | fwd_pkts_tot | bwd_pkts_tot | fwd_data_pkts_tot | bwd_data_pkts_tot | ... | active.std | idle.min | idle.max | idle.tot | idle.avg | idle.std |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 38667 | 1883 | tcp | mqtt | 32.011598 | 9 | 5 | 3 | 3 | ... | 0.0 | 2.972918e+07 | 2.972918e+07 | 2.972918e+07 | 2.972918e+07 | 0.0 |
| 1 | 1 | 51143 | 1883 | tcp | mqtt | 31.883584 | 9 | 5 | 3 | 3 | ... | 0.0 | 2.985528e+07 | 2.985528e+07 | 2.985528e+07 | 2.985528e+07 | 0.0 |
| 2 | 2 | 44761 | 1883 | tcp | mqtt | 32.124053 | 9 | 5 | 3 | 3 | ... | 0.0 | 2.984215e+07 | 2.984215e+07 | 2.984215e+07 | 2.984215e+07 | 0.0 |
| 3 | 3 | 60893 | 1883 | tcp | mqtt | 31.961063 | 9 | 5 | 3 | 3 | ... | 0.0 | 2.991377e+07 | 2.991377e+07 | 2.991377e+07 | 2.991377e+07 | 0.0 |
| 4 | 4 | 51087 | 1883 | tcp | mqtt | 31.902362 | 9 | 5 | 3 | 3 | ... | 0.0 | 2.981470e+07 | 2.981470e+07 | 2.981470e+07 | 2.981470e+07 | 0.0 |

5 rows x 85 columns

| Phase | Activity | Code and Output |
|---|---|---|
| **Transform** | 1. Data Cleaning: Remove missing values and duplicate records. | **Code:** df.dropna(inplace=True), df.drop_duplicates(inplace=True) |
| | 2. Feature Selection: Remove unnecessary index columns (e.g., 'Unnamed: 0'). | **Code:** if 'Unnamed: 0' in df.columns: df.drop(columns=['Unnamed: 0'], inplace=True) |
| | 3. Categorical Encoding: Convert 'proto' and 'service' into numerical format using LabelEncoder. | **Code:** le = LabelEncoder(), df['proto_encoded'] = le.fit_transform(df['proto']) |
| | 4. Final Verification: Confirm the new dimensions of the dataset. | **Output:** Transformation Complete. New Shape: (123117, 87). |

| Phase 2 Output |
| --- |
| `Transformation Complete. New Shape: (123117, 87)` |

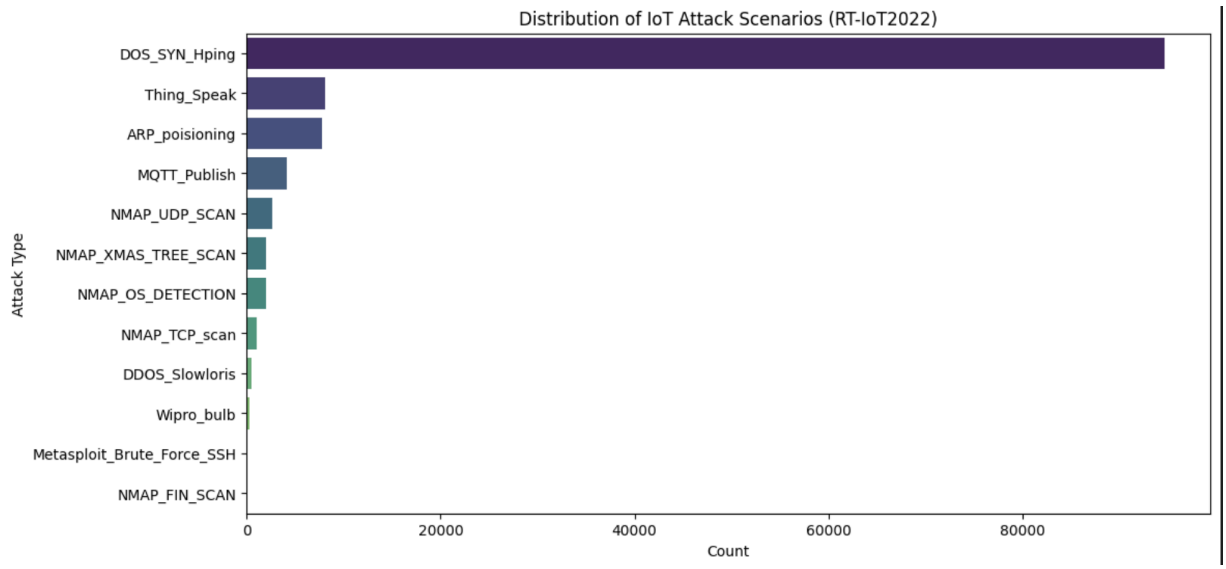
| Phase | Activity | Code and Output |
| --- | --- | --- |
| **Load** | 1. Statistical Analysis: Generate descriptive statistics for key network features. | **Code:** df[['flow_duration', 'fwd_pkts_tot', ...]].describe() |
| | 2. Summary Statistics: Output mean, standard deviation, and range for traffic duration. | **Output:** flow_duration mean: 3.80, std: 130.00, max: 21728.33. |
| | 3. Data Visualization: Create a horizontal count plot of the 9 attack scenarios and normal patterns. | **Code:** sns.countplot(data=df, y='Attack_type', palette='viridis') |
| | 4. Visual Output: Display the distribution of IoT attack scenarios. | **Output:** A plot showing Attack Types like MQTT_Publish, DOS_SYN_Hping. |

| Phase 3 Output |
| --- |

```
--- Statistical Summary of Network Traffic ---
       flow_duration   fwd_pkts_tot   bwd_pkts_tot  fwd_data_pkts_tot
count  123117.000000  123117.000000  123117.000000      123117.000000
mean        3.809566       2.268826       1.909509           1.471218
std       130.005408      22.336565      33.018311          19.635196
min         0.000000       0.000000       0.000000           0.000000
25%         0.000001       1.000000       1.000000           1.000000
50%         0.000004       1.000000       1.000000           1.000000
75%         0.000005       1.000000       1.000000           1.000000
max     21728.335578    4345.000000   10112.000000        4345.000000
/var/folders/q7/1g4_b9fn3pd7z_qq7345v6vr0000gn/T/ipykernel_8553/3056271843.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(data=df, y='Attack_type', order=df['Attack_type'].value_counts().index, palette='viridis')
```




## Summarization

I completed all phase processes on the RT-IoT2022 dataset. First, I used the Kaggle API to download the latest version of the data directly into the environment. IDuring the Transformation phase, I had to spend a lot of time cleaning the data. I found a bunch of duplicates and null values that would've messed up the results, so I cleared those out. The hardest part was dealing with the categorical columns like proto and service. Since you can't run math on words, I used LabelEncoder to swap them into numbers so the statistical functions wouldn't crash. Lastly, I loaded the data to perform statistical analysis and created a visualization to see which network behaviors were most common.

**Insights Derived:**

- **Attack Prevalence:** The dataset is heavily populated with specific attack types like MQTT_Publish and DOS_SYN_Hping. This highlights how frequently these methods are used to target IoT infrastructure.
- **Traffic Characteristics:** The flow_duration analysis shows that the majority of network flows are extremely short (median near 0.000004s), which is a common characteristic of automated botnet attacks or rapid scanning.
- **Dataset Robustness:** With over 123,000 records and 80+ features, this dataset provides a very detailed look at both normal IoT behavior and malicious activities, making it highly effective for training security monitoring tools.